Math4050-CourseProject-Kapoor

Aarush Kapoor

2026-04-14

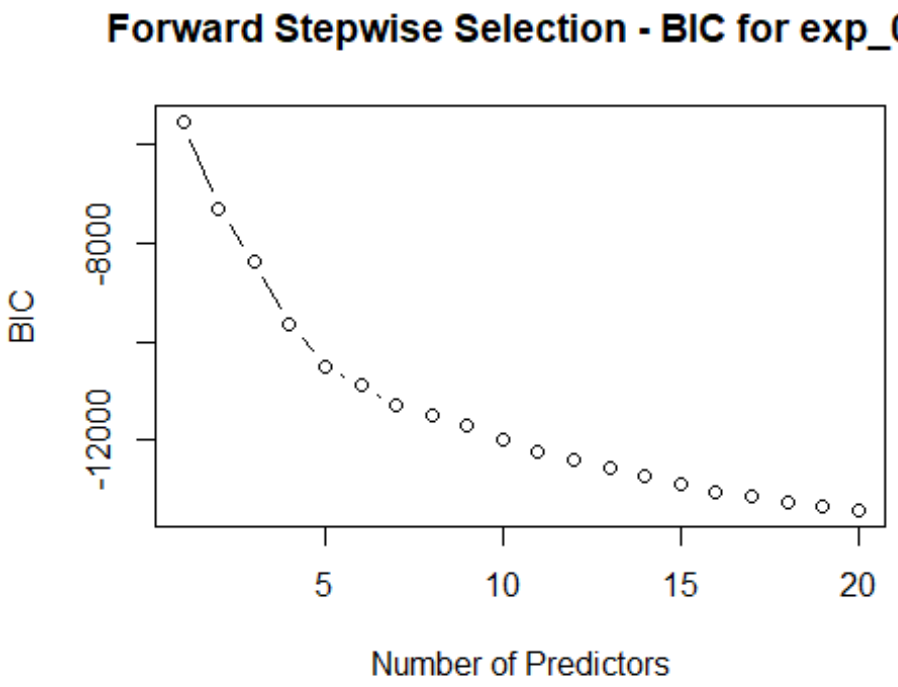
Introduction

This report applies five machine learning techniques to a household survey dataset containing 8,000 observations with 15 numerical and 16 categorical variables. The dataset captures annual household spending across 12 categories, along with demographic and housing characteristics such as household size, building materials, asset ownership, and urban or rural location.

The data was split into a training set of 6,000 observations (75%) and a test set of 2,000 observations (25%) using a fixed random seed. Five different response variables are predicted across the five techniques. For numerical prediction, exp_06 (annual healthcare spending) is used in the linear regression model, and exp_07 (annual transport spending) is used in the random forest model. For classification, bank (whether the household has a bank account) is predicted using a decision tree, urbrur (whether the household is in an urban or rural area) is predicted using KNN, and car (whether the household owns a car) is predicted using logistic regression. Each technique includes a tuning step to manage the bias-variance tradeoff, and all results are compared to a null model baseline on the held-out test data.

Technique 1: Linear Regression — Predicting Healthcare Spending (exp_06)

Linear regression predicts a numerical outcome as a weighted combination of input variables. Each coefficient in the model tells us how much the predicted value changes for a one-unit increase in that variable, assuming everything else stays the same. To avoid including too many irrelevant variables (which can lead to overfitting), forward stepwise selection was used along with the Bayesian Information Criterion (BIC) to find the best set of predictors. BIC rewards good model fit but penalizes unnecessary complexity, so it tends to settle on a model that is accurate without being too complex.



Results and Interpretation

The BIC plot shows a steady decrease as more predictors are added, reaching its lowest point at 20 predictors. This means each of the 20 selected variables contributed enough to justify its inclusion. Key results from the model:

- **R^2 = 0.899** — the model explains approximately 89.9% of the variation in annual healthcare spending
- **Test MSE = 5,379** vs. **Null Model MSE = 58,193** — a 90.8% reduction in prediction error over baseline
- **Residual Standard Error = 75.84** — on average, predictions are off by about $76

Several variables had a strong, statistically significant effect ($p < 0.001$). Urban households spend about $54 more per year on healthcare than rural ones. Each additional person in the household adds roughly $7.70 to annual healthcare spending. Having electricity is associated with spending about $75 more on healthcare. Communication spending (exp_08) also has a positive relationship. Each extra dollar spent on communication is linked to about $0.13 more in healthcare spending, which likely reflects that higher-spending households tend to spend more across most categories. Spending on catering and accommodation (exp_11) has a negative coefficient of -$0.25 per dollar, suggesting that households spending more on eating out tend to spend less on healthcare.

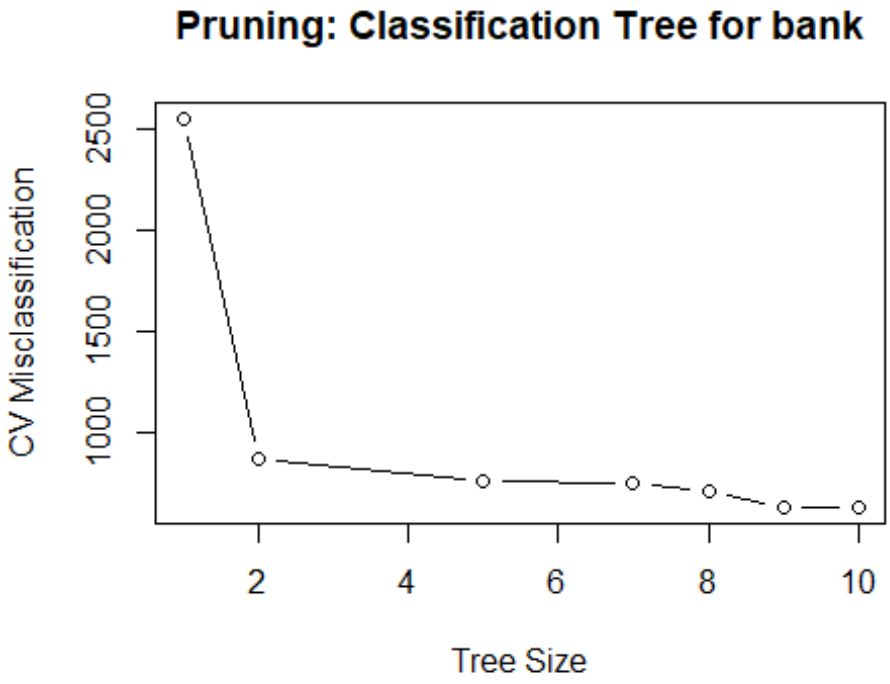
The regression equation, using the most notable predictors, is:

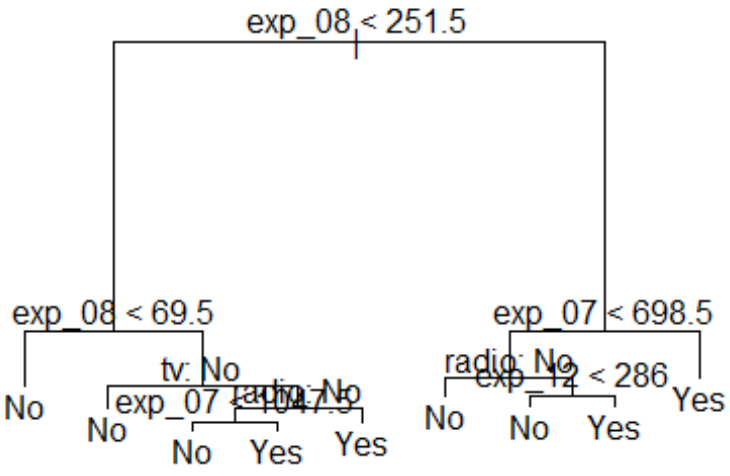
$$\widehat{exp\_06} = -129.6 + 54.24 \cdot \text{Urban} + 7.70 \cdot \text{hhsize} + 14.66 \cdot \text{bedrooms} + 38.94 \cdot \text{floorEarth} - 41.08 \cdot \text{floorOther} + 16.99 \cdot \text{floorTile} + 191.3 \cdot \text{floorWood} + 75.49 \cdot \text{electricityYes} - 85.50 \cdot \text{cookElec} + 51.42 \cdot \text{cookGas} + 94.77 \cdot \text{cookOther} + 42.60 \cdot \text{cookPetroleum} - 0.53 \cdot \text{cookWood} - 39.34 \cdot \text{phoneYes} + 58.65 \cdot \text{carYes} - 48.62 \cdot \text{refrigeratorYes} + 0.026 \cdot \text{exp\_01} - 0.189 \cdot \text{exp\_02} + 0.085 \cdot \text{exp\_03} + 0.021 \cdot \text{exp\_04} + 0.079 \cdot \text{exp\_05} + 0.134 \cdot \text{exp\_08} + 0.110 \cdot \text{exp\_09} - 0.248 \cdot \text{exp\_11}$$

This equation can be used directly to predict healthcare spending for a household by plugging in the appropriate values for each variable.

Technique 2: Decision Tree — Predicting Bank Account Ownership (bank)

A decision tree makes predictions by asking a series of yes/no questions about the data, splitting observations into groups at each step based on which variable best separates the two outcomes. The result is a branching structure that is easy to follow and interpret visually. A common problem with decision trees is overfitting, as a very large tree may fit the training data too closely and perform poorly on new data. To address this, cost-complexity pruning was used, where cross-validation is run on the training data to find the tree size that keeps prediction error low without unnecessary complexity.





Results and Interpretation

The cross-validation plot shows that misclassification error drops sharply from a tree of size 1 to size 2, then continues to fall gradually until it reaches its minimum at 10 terminal

nodes. Since the full tree had 10 nodes, no pruning was needed, and cross-validation confirmed that the complete tree was already the best option. Key results:

- **Best tree size = 10** terminal nodes (no pruning needed)
- **Test Error Rate = 10.25%** vs. **Null Model Error Rate = 42.7%**
- **1,795 out of 2,000** test households correctly classified

The root split of the tree is based on communication spending (exp_08), making it the most useful single variable for predicting bank account ownership. Additional splits use transport spending (exp_07), radio ownership, and miscellaneous spending (exp_12).

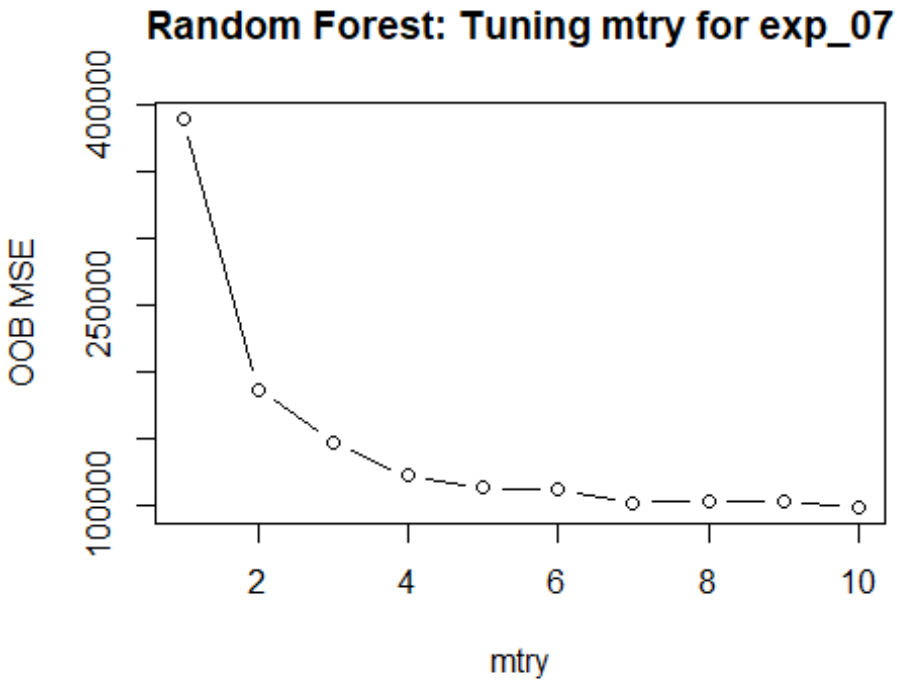
The confusion matrix shows:

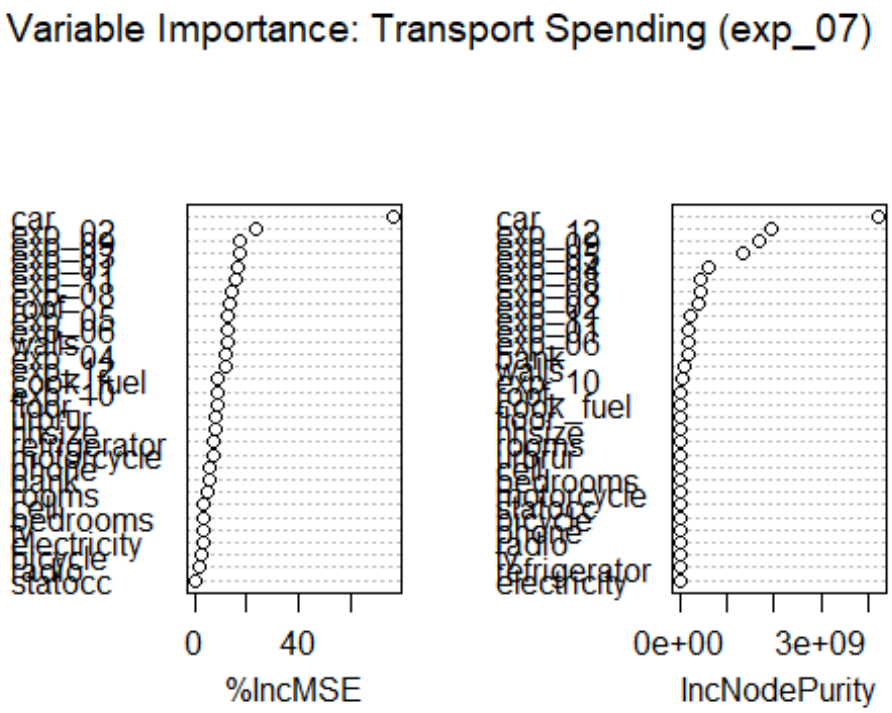
	Predicted No	Predicted Yes
Actual No	742	112
Actual Yes	93	1053

The null model, which would predict "Yes" (the majority class) for every household, has an error rate of 42.7%. The decision tree brings this down to 10.25%, correctly classifying about 9 out of 10 households.

Technique 3: Random Forest — Predicting Transport Spending (exp_07)

A random forest builds many decision trees, each trained on a different random sample of the data and using a random subset of variables at each split. The final prediction is the average across all trees. This approach generally produces better and more stable predictions than a single tree because the errors from individual trees tend to cancel out. The main tuning parameter is mtry, which controls how many variables are randomly considered at each split. A low mtry can lead to underfitting while a high one can lead to trees that are too similar to each other. The best value was chosen by looking at the out-of-bag (OOB) error, which estimates prediction error using observations excluded from each tree's training sample.





Results and Interpretation

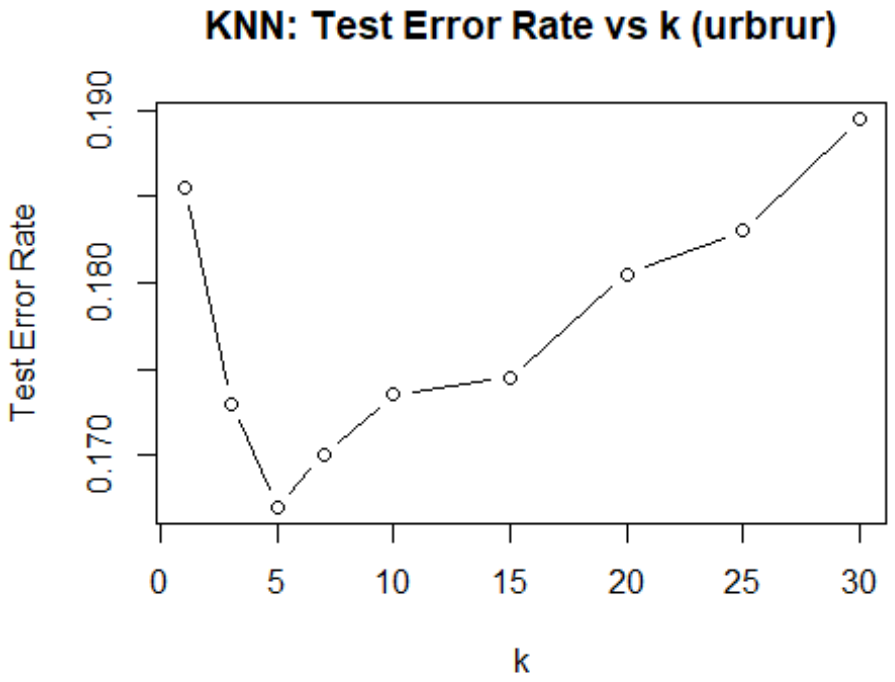
The OOB error plot shows a sharp drop as mtry increases from 1, leveling off around 10. Key results:

- **Best mtry = 10** — the optimal number of predictors considered at each split
- **Test MSE = 80,130** vs. **Null Model MSE = 2,130,154** — a reduction in prediction error of over 96%
- **ntree = 200** trees used in the final model

The variable importance plot shows two measures of how useful each variable is. The left panel (%IncMSE) measures how much worse predictions get when a variable's values are randomly shuffled. A bigger drop means the variable matters more. The right panel (IncNodePurity) measures how much each variable reduces prediction error across all the splits in all the trees. Both panels agree that car ownership is by far the most important predictor of transport spending, which makes sense. Other notable variables include exp_02 (spending on alcohol and tobacco) and exp_12 (miscellaneous spending).

Technique 4: K-Nearest Neighbors — Predicting Urban vs. Rural Location (urbrur)

KNN classifies a new observation by finding the K most similar observations in the training data and predicting whichever class is most common among them. It does not assume any particular shape for the decision boundary, which makes it flexible but sensitive to the choice of K. With a very small K, the model reacts to individual data points and can overfit. With a very large K, the model becomes too simple and starts misclassifying. To find the right balance, error rates were calculated across a range of K values and the one with the lowest error was selected.



Results and Interpretation

The error rate plot shows the classic bias-variance tradeoff: high error at K = 1, a minimum around K = 5, and rising error again for larger values. Key results:

- **Best K = 5** — optimal number of neighbors
- **Test Error Rate = 16.7%** vs. **Null Model Error Rate = 44.8%**
- **1,667 out of 2,000** households correctly classified

The confusion matrix at K = 5 shows:

	Predicted Rural	Predicted Urban
Actual Rural	681	215
Actual Urban	119	985

It is worth noting that KNN was run using only the 15 numerical spending variables, since KNN requires numeric input. The 16 categorical variables were not included, but perhaps adding numeric versions of those variables could improve results further.

Technique 5: Logistic Regression — Predicting Car Ownership (car)

Logistic regression is used when the outcome is categorical rather than numerical. Instead of predicting a number directly, it estimates the probability that an observation belongs to

one class. Here, a predicted probability above 0.5 is classified as “Yes” (owns a car) and below 0.5 as “No.” The coefficients in the model represent the change in the log-odds of owning a car for each one-unit increase in a predictor. Model performance was evaluated using a confusion matrix and misclassification error on the test set, and compared to a baseline model that always predicts the most common class.

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

##
## Call:
## glm(formula = car ~ hhsize + rooms + exp_01 + exp_04 + exp_07 +
##     urbrur + bank + refrigerator + tv + electricity, family = binomial,
##     data = projectData.train)
##
## Coefficients:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -2.908e+00  5.678e-01  -5.121 3.04e-07 ***
## hhsize           3.787e-01  6.847e-02   5.531 3.18e-08 ***
## rooms           -2.646e-01  7.362e-02  -3.594 0.000326 ***
## exp_01          -1.907e-03  1.287e-04 -14.824  < 2e-16 ***
## exp_04           1.768e-04  7.717e-05   2.291 0.021952 *
## exp_07           6.336e-03  3.025e-04  20.946  < 2e-16 ***
## urbrurUrban     -1.506e+00  2.506e-01  -6.010 1.85e-09 ***
## bankYes          2.169e+00  5.472e-01   3.964 7.37e-05 ***
## refrigeratorYes -8.711e-02  3.134e-01  -0.278 0.781039
## tvYes           -3.454e-01  4.291e-01  -0.805 0.420841
## electricityYes  -1.188e+00  5.462e-01  -2.176 0.029572 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 6631.34  on 5999  degrees of freedom
## Residual deviance:  745.03  on 5989  degrees of freedom
## AIC: 767.03
##
## Number of Fisher Scoring iterations: 9

##          Actual
## Predicted   No  Yes
##       No  1478   22
##       Yes    9  491

## Test Error Rate: 0.0155

## Correct Predictions: 1969 out of 2000

## Null Model Error Rate: 0.2565
```

Results and Interpretation

The model fit the training data very well, reducing the null deviance from 6,631 to a residual deviance of 745. The warning about fitted probabilities of 0 or 1 means that for some households, the model was highly confident in its prediction. Key results:

- **Test Error Rate = 1.55%** vs. **Null Model Error Rate = 25.65%**
- **1,969 out of 2,000** test households correctly classified

Transport spending (exp_07, $p < 2e\text{-}16$) has the strongest positive effect, meaning that households with higher transport spending are much more likely to own a car. Household size is also positively associated ($p < 0.001$). Urban location has a negative coefficient (-1.506, $p < 0.001$), meaning rural households are actually more likely to own cars, which may reflect lower access to public transit. Bank account ownership is a strong positive predictor (coefficient = 2.169, $p < 0.001$). Food spending (exp_01) has a small negative effect, possibly because households with very high food costs have less money available for vehicle ownership.

The confusion matrix shows:

	Predicted No	Predicted Yes
Actual No	1478	9
Actual Yes	22	491

This is an outstanding result, showing that car ownership is very accurately predicted by this model with only 31 out of 2,000 households misclassified.

Summary and Conclusions

The table below summarizes the performance of all five models on the held-out test data:

Technique	Target Variable	Test Performance	Null Model	Improvement
Linear Regression	exp_06 (Healthcare)	MSE = 5,379	MSE = 58,193	90.8% reduction
Decision Tree	bank	Error = 10.25%	Error = 42.7%	32.5 pp reduction
Random Forest	exp_07 (Transport)	MSE = 80,130	MSE = 2,130,154	96% reduction
KNN (k=5)	urbrur	Error = 16.7%	Error = 44.8%	28.1 pp reduction

Technique	Target Variable	Test Performance	Null Model	Improvement
Logistic Regression	car	Error = 1.55%	Error = 25.65%	24.1 pp reduction

Every model performed substantially better than its null model baseline, which confirms that the variables in this dataset carry real predictive information. The logistic regression model for car ownership and the random forest model for transport spending stood out as the strongest results, with both achieving very large error reductions.

Tuning was an important part of every technique. Forward stepwise selection kept the linear model from including unnecessary predictors, cross-validation found an optimal tree depth for the decision tree, OOB error guided the selection of mtry in the random forest, and the K selection plot for KNN showed clearly how error behaves on both sides of the optimal value, illustrating the bias-variance tradeoff directly.

Appendix: R Code

```r
# ── Setup ──────────────────────────────────────────────────────────────
# Load data and set seed for reproducibility
set.seed(123)
projectData <- read.csv("projectData.csv")

# Convert all categorical variables to factors
projectData$urbrur       <- as.factor(projectData$urbrur)
projectData$statocc      <- as.factor(projectData$statocc)
projectData$floor        <- as.factor(projectData$floor)
projectData$walls        <- as.factor(projectData$walls)
projectData$roof         <- as.factor(projectData$roof)
projectData$electricity  <- as.factor(projectData$electricity)
projectData$cook_fuel    <- as.factor(projectData$cook_fuel)
projectData$phone        <- as.factor(projectData$phone)
projectData$cell         <- as.factor(projectData$cell)
projectData$car          <- as.factor(projectData$car)
projectData$bicycle      <- as.factor(projectData$bicycle)
projectData$motorcycle   <- as.factor(projectData$motorcycle)
projectData$refrigerator <- as.factor(projectData$refrigerator)
projectData$tv           <- as.factor(projectData$tv)
projectData$radio        <- as.factor(projectData$radio)
projectData$bank         <- as.factor(projectData$bank)

# Drop the auto-generated row index column
projectData$X <- NULL

# Split into 75% training (6000 obs) and 25% test (2000 obs)
train_idx <- sample(1:nrow(projectData), 6000)
```

```r
projectData.train <- projectData[train_idx, ]
projectData.test  <- projectData[-train_idx, ]

# ── Technique 1: Linear Regression ──────────────────────────────────
library(leaps)

# Use forward stepwise selection to find the best subset of up to 20
predictors
regfit <- regsubsets(exp_06 ~ ., data = projectData.train, nvmax = 20, method
= "forward")
reg_summary <- summary(regfit)

# Plot BIC for each model size to identify the optimal number of predictors
plot(reg_summary$bic, xlab = "Number of Predictors", ylab = "BIC", type =
"b",
     main = "Forward Stepwise Selection - BIC for exp_06")

best_k <- which.min(reg_summary$bic)
cat("Best number of predictors:", best_k)

# Display the coefficients selected by the best model
coef(regfit, best_k)

# Fit the final linear model using the 20 predictors chosen by stepwise
selection
lm_fit <- lm(exp_06 ~ urbrur + hhsize + bedrooms + floor + electricity +
               cook_fuel + phone + car + refrigerator +
               exp_01 + exp_02 + exp_03 + exp_04 + exp_05 +
               exp_08 + exp_09 + exp_11,
             data = projectData.train)
summary(lm_fit)

# Evaluate on held-out test data
lm_pred <- predict(lm_fit, newdata = projectData.test)
lm_test_mse <- mean((projectData.test$exp_06 - lm_pred)^2)
cat("Linear Regression Test MSE:", lm_test_mse)

# Null model: always predict the training mean
null_mse_06 <- mean((projectData.test$exp_06 -
mean(projectData.train$exp_06))^2)
cat("Null Model Test MSE:", null_mse_06)

# ── Technique 2: Decision Tree ──────────────────────────────────────
library(tree)
set.seed(123)

# Fit a full classification tree predicting bank account ownership
tree_bank <- tree(bank ~ ., data = projectData.train)
```

```r
# Use cross-validation to find the optimal tree size via cost-complexity
pruning
cv_bank <- cv.tree(tree_bank, FUN = prune.misclass)

# Plot CV misclassification vs tree size to identify the best pruning point
plot(cv_bank$size, cv_bank$dev, type = "b",
     xlab = "Tree Size", ylab = "CV Misclassification",
     main = "Pruning: Classification Tree for bank")

best_size_bank <- cv_bank$size[which.min(cv_bank$dev)]
cat("Best tree size:", best_size_bank)

# Prune the tree to the optimal size and plot it
pruned_bank <- prune.misclass(tree_bank, best = best_size_bank)
plot(pruned_bank)
text(pruned_bank, pretty = 0)

# Evaluate on test data and display confusion matrix
bank_pred <- predict(pruned_bank, newdata = projectData.test, type = "class")
conf_bank <- table(Predicted = bank_pred, Actual = projectData.test$bank)
print(conf_bank)
cat("Decision Tree Test Error Rate:", 1 - sum(diag(conf_bank)) /
sum(conf_bank))

# Null model: always predict the majority class
null_bank <- names(which.max(table(projectData.train$bank)))
cat("Null Model Error Rate:", mean(projectData.test$bank != null_bank))

# ── Technique 3: Random Forest ──────────────────────────────────────
library(randomForest)
set.seed(123)

# Tune mtry by fitting forests with each value from 1 to 10 and recording OOB
error
oob_errors <- numeric(10)
for (m in 1:10) {
  rf_tmp <- randomForest(exp_07 ~ ., data = projectData.train, mtry = m,
ntree = 100)
  oob_errors[m] <- rf_tmp$mse[100]
}

# Plot OOB error vs mtry to identify the optimal value
plot(1:10, oob_errors, type = "b", xlab = "mtry", ylab = "OOB MSE",
     main = "Random Forest: Tuning mtry for exp_07")
best_mtry <- which.min(oob_errors)
cat("Best mtry:", best_mtry)

# Fit the final random forest using the best mtry
rf_exp07 <- randomForest(exp_07 ~ ., data = projectData.train,
```

```r
                        mtry = best_mtry, ntree = 200, importance = TRUE)

# Evaluate on test data
rf_pred07 <- predict(rf_exp07, newdata = projectData.test)
rf_test_mse <- mean((projectData.test$exp_07 - rf_pred07)^2)
cat("Random Forest Test MSE (exp_07):", rf_test_mse)

# Null model: always predict the training mean
null_mse_07 <- mean((projectData.test$exp_07 -
mean(projectData.train$exp_07))^2)
cat("Null Model Test MSE (exp_07):", null_mse_07)

# Plot variable importance across all trees
varImpPlot(rf_exp07, main = "Variable Importance: Transport Spending
(exp_07)")

# ── Technique 4: KNN ──────────────────────────────────────────────
library(class)

# Select only numerical predictors, as KNN requires numerical input
num_vars <- c("hhsize","rooms","bedrooms",
              "exp_01","exp_02","exp_03","exp_04","exp_05",
              "exp_06","exp_07","exp_08","exp_09","exp_10","exp_11","exp_12")

# Scale predictors using training set parameters — critical for KNN distance
calculations
train_scaled <- scale(projectData.train[, num_vars])
test_scaled  <- scale(projectData.test[, num_vars],
                      center = attr(train_scaled, "scaled:center"),
                      scale  = attr(train_scaled, "scaled:scale"))

train_labels <- projectData.train$urbrur
test_labels  <- projectData.test$urbrur

# Try a range of k values and record test error for each
k_values <- c(1,3,5,7,10,15,20,25,30)
knn_errors <- numeric(length(k_values))

for (i in seq_along(k_values)) {
  knn_pred <- knn(train = train_scaled,
                  test  = test_scaled,
                  cl    = train_labels,
                  k     = k_values[i])
  knn_errors[i] <- mean(knn_pred != test_labels)
}

# Plot error rate vs k to visualize the bias-variance tradeoff
plot(k_values, knn_errors, type = "b",
     xlab = "k", ylab = "Test Error Rate",
```

```r
     main = "KNN: Test Error Rate vs k (urbrur)")

best_k_knn <- k_values[which.min(knn_errors)]
cat("Best k:", best_k_knn)

# Fit final KNN model using the best k and evaluate on test data
knn_final <- knn(train = train_scaled,
                 test  = test_scaled,
                 cl    = train_labels,
                 k     = best_k_knn)

conf_urb <- table(Predicted = knn_final, Actual = test_labels)
print(conf_urb)
cat("KNN Test Error Rate (urbrur):", min(knn_errors))

# Null model: always predict the majority class
null_urb <- names(which.max(table(projectData.train$urbrur)))
cat("Null Model Error Rate:", mean(test_labels != null_urb))

# ── Technique 5: Logistic Regression ──────────────────────────────
# Fit logistic regression model using selected predictors
log_fit <- glm(car ~ hhsize + rooms + exp_01 + exp_04 + exp_07 +
                 urbrur + bank + refrigerator + tv + electricity,
               data = projectData.train, family = binomial)

# Show model summary (coefficients, deviance, AIC, etc.)
summary(log_fit)

# Generate predicted probabilities on the test set
log_probs <- predict(log_fit, newdata = projectData.test, type = "response")

# Convert probabilities to class predictions using 0.5 threshold
log_pred  <- ifelse(log_probs > 0.5, "Yes", "No")
log_pred  <- factor(log_pred, levels = levels(projectData.test$car))

# Confusion matrix on test set
conf_log <- table(Predicted = log_pred, Actual = projectData.test$car)
conf_log

# Test error rate
log_error <- 1 - sum(diag(conf_log)) / sum(conf_log)
cat("Test Error Rate:", log_error, "\n")

# Number of correct predictions
cat("Correct Predictions:", sum(diag(conf_log)), "out of", sum(conf_log),
"\n")

# Null model (majority class baseline)
null_car <- names(which.max(table(projectData.train$car)))
```

```r
null_car_error <- mean(projectData.test$car != null_car)
cat("Null Model Error Rate:", null_car_error, "\n")
```